

Climate Crusade

SENG 401 - Software Architecture

Group 4: SAAGEE Studios



Creators:

Ebitimi Dambo , Arol Nokam Wafo, Stephan Agwai
Aadil Bashir, Gauthier Appaix, Idoro Emmanuel

Date: 13th January 2026

Institution: University of Calgary

Table Of Contents

Game Concept Overview	3
Sustainability Goal Alignment	3
Overall Game Flow	4
Scene Architecture	6
Concept Art & Visual Design	10
Gameplay Mechanics Overview	11
Functional Requirements	12

Game Concept Overview

Climate Crusade is a serious game centered around the United Nations' 13th Sustainable Development Goal, Climate Action. The game explores the effects of climate change on the planet and encourages players to learn more about these impacts, as well as the actions that can be taken to prevent or reduce them. The target audience consists of individuals who are open to change and willing to engage with environmental issues. Through interactive gameplay, Climate Crusade aims to motivate players to take greater responsibility for protecting the planet.

The game is designed to take players on a journey through different locations and biomes, exploring the effects that climate change has had on these environments and engaging in activities aimed at stopping or reversing these impacts. To achieve this, Climate Crusade is structured as a level-based game, allowing players to become immersed in the story each level presents. Players control the game's mascot, Alvin, and help him overcome carefully designed obstacles such as platforming challenges, dig-sites, shooter-style segments, and other interactive mechanics, all of which contribute to the overall goal of saving the Earth. Throughout each level, players can collect stars, which are converted into health points that contribute to the Earth's overall health status. Once all levels have been completed and all stars have been collected, the Earth is fully restored and the game is completed.

Sustainability Goal Alignment

Climate Crusade aims to explore climate change and the actions that can be taken to stop or potentially reverse its effects. The game levels are designed around three main guiding points, applied in the following order:

- the selected location or biome
- the effects of climate change on that environment
- the actions that can be taken to stop or mitigate those effects

Each level is developed by first selecting a specific location. Once the location is chosen, research is conducted on how climate change impacts that environment, followed by discussions on ways in which these effects can be reduced, reversed, or prevented. Climate Crusade explores a variety of climate change effects, including:

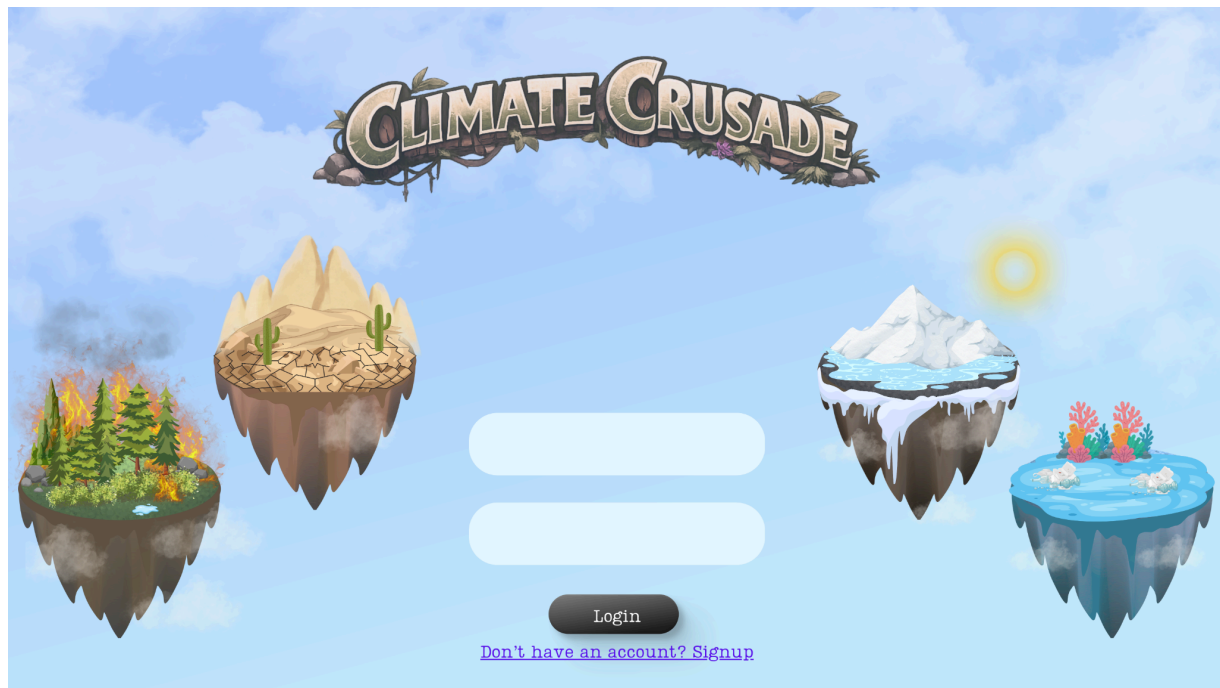
- Acid rain, caused by greenhouse gas emissions being absorbed into clouds
- Water pollution, caused by waste being carried during extreme weather events
- Extreme desert dryness, caused by prolonged exposure to intense solar radiation
- Ozone layer depletion, caused by debris and pollutants in the upper atmosphere
- Melting tundras, caused by rising temperatures and ozone layer depletion

By exploring these effects and their causes, the game aims to raise awareness among players and encourage them to consider lifestyle changes that support Climate Action and contribute to the overall health of the Earth.

Overall Game Flow

The game begins with a title screen where the player is required to enter their login credentials in order to proceed. Once the entered details are validated, a Play button is displayed. The game will only progress beyond the title screen after the Play button has been selected.

If the player is new and does not have an existing account, they are given the option to sign up by clicking a hyperlink located beneath the login button. Selecting this option prompts a registration form to appear, requesting the necessary information required to create a new account.



Once the Play button is selected, a cutscene may be triggered depending on whether it is the player's first time launching the game. This introductory cutscene is used to establish the narrative context and provide the player with a clear sense of purpose. The cutscene is brief and introduces the story of Alvin and his people, explaining how he arrived on Earth and why he is involved in protecting it.

After the cutscene concludes, the player is taken to the level selection screen. From this screen, the player can view the Earth's current health status as well as information related to each available level and its associated environmental challenge.

The level selection screen will contain slots for six playable levels, with each level representing the contribution of one team member. Each level slot displays the number of stars the player has collected for that specific level, allowing players to easily track their progress.

When a level is selected, additional information is presented to the player, including the location or biome, the climate change issue being addressed, the effects associated with that issue, and instructions on how to complete the level.

Once a level is selected, a short cutscene will play that follows a location-specific mascot and highlights a major cause of climate change relevant to that environment, such as greenhouse gas emissions, along with its resulting effects. This cutscene is intended to set the scene for the level and fully immerse the player in the experience.

After the cutscene concludes, the player gains control of Alvin and must guide him through the various obstacles and challenges presented within the level. Where possible, each level will include narration delivered in the native language of the location in which it is set, with English subtitles available to ensure accessibility.

At the end of each level, the player is congratulated and provided with additional tips on how similar climate-related disasters can be prevented in real life. This approach helps balance engaging gameplay with educational content, creating an experience that is both informative and enjoyable.

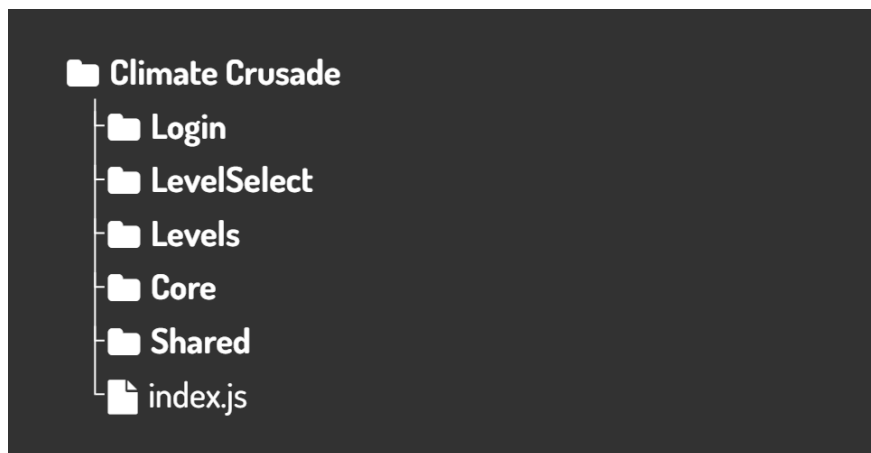
Game progress is stored in an SQL database and is retrieved upon startup. The primary data stored includes the highest number of stars collected for each level. Once all levels have been completed and all collectible stars have been retrieved, the Earth's health will be restored to 100%, starting from an initial value of 50%. At this point, the game is considered officially complete.

Scene Architecture

This section discusses how scenes will be structured in Phaser.js. Each team member will be developing their level on their own system, so this structure is intended to ensure that the game can be assembled smoothly at the end of the development cycle without requiring developers to micromanage one another's level creation processes.

The game is structured using a modular scene-based architecture to support parallel development and easy integration of independently developed levels. Core game flow components such as login and level selection are separated from gameplay content, while each level is implemented as a self-contained module with a standardized entry point that each developer must follow. This design allows individual developers to work independently without requiring changes to shared code during integration.

Top Level Folder Structure:



Each folder represents a distinct responsibility within the system and interacts with other components only through well-defined interfaces.

	Component	Purpose
1.	Login	The Login module is responsible for user authentication and session initialization. It serves as the first point of interaction when the game starts.
2.	LevelSelect	The Level Selection module provides navigation between available levels. It displays level information retrieved from configuration files and allows the player to choose which level to play.
3.	Levels	The Levels directory contains all playable game levels. Each level is implemented as a self-contained module with its own assets, scenes, and configuration file. Levels are designed to be independent of one another
4.	Core	The Core module coordinates transitions between major parts of the game, including login, level selection, and gameplay. It manages level discovery, loading, and transitions.

5.	Shared	The Shared directory contains reusable assets and utilities that are used across multiple modules, such as UI components, global assets, and constants.
----	--------	---

Level Architecture:



	Component	Purpose
1.	level.config.json	This file defines metadata required to discover and load the level, including its identifier, display name, entry scene, and optional cutscene information. It serves as a contract between the level and the core game system.
2.	EntryScene.js	EntryScene.js is the single guaranteed entry point for the level and is the only file loaded directly by the game system. It represents the first playable scene and may internally transition to other scenes within the level.
3.	Scenes	This directory contains all internal scenes used by the level. These scenes are not accessed externally and may be organized freely by the level developer.
4.	Assets	The Assets directory contains all level-specific assets, ensuring isolation and preventing naming conflicts with other levels.
5.	Cutscene	This optional directory contains cutscene scenes used to introduce the level. Cutscenes are declared explicitly in the level configuration file and are handled by the core game flow.

Runtime Interactions and Scene Flow

This section describes how the different parts of the system interact at runtime, from application startup to level execution. The purpose of this design is to allow game flow components (login and level selection) to load any level without requiring knowledge of how that level is internally implemented.

Step 1: Application Startup

When the game launches, execution begins in the main entry file `index.js`. This file initializes Phaser and sets the starting scene to the login scene: `Login/LoginScene.js`.

Step 2: Login Completion

Once the user successfully logs in or signs up, `LoginScene.js` does not load any levels directly. Instead, it signals the Core layer: `Core/GameFlowManager.js` e.g `GameFlowManager.goToLevelSelect()`. This keeps login logic isolated and prevents it from depending on gameplay or level code.

Step 3: Level Selection Scene

The Core layer transitions the game to: `LevelSelect/LevelSelectScene.js`. When this scene is created, it does not hard-code any levels. Instead, it asks the Core layer for available levels in `Core/LevelRegistry.js`. The Level Registry scans the `Levels/` directory and reads each level's configuration file:

- `Levels/Level01/level.config.json`
- `Levels/Level02/level.config.json`
- `Levels/Level03/level.config.json`
- `Levels/Level04/level.config.json`

Each configuration file provides metadata such as:

- Level ID
- Display name
- Entry scene file
- Whether the level has a cutscene

The Level Selection scene uses this metadata to display level slots, star counts, and level information.

Step 4: Level Selection by the Player

When the player selects a level, the Level Selection scene does not import or load the level itself. Instead, it passes the selected level ID to the Core layer: `GameFlowManager.startLevel("level01")`. From this point onward, all loading responsibility belongs to the Core.

Step 5: Level Loading via Core

The Core layer retrieves the corresponding configuration file: `Levels/Level01/level.config.json`. Using the information in this file, the Core determines:

- Whether a cutscene exists
- What the entry scene file is

If a cutscene is defined, the Core loads: Levels/Level01/Cutscene/IntroScene.js. Once the cutscene completes, or immediately if no cutscene exists, the Core loads the level's guaranteed entry point: Levels/Level01/EntryScene.js. This is the only file the system ever loads directly for a level.

Step 6: Internal Level Execution

From EntryScene.js, the level takes full control of its internal flow. Inside the level folder, the developer may transition between scenes such as:

- Levels/Level01/Scenes/PuzzleScene.js
- Levels/Level01/Scenes/CombatScene.js
- Levels/Level01/Scenes/BossScene.js

These internal scenes can be structured freely by the level developer. At this point, the level is a fully self-contained system.

Step 7: Level Completion

When the level is completed, the level signals completion back to the Core: GameFlowManager.onLevelComplete(levelId). The Core then determines the next state of the game, such as:

- Returning to the Level Selection scene
- Updating stored progress in the database
- Triggering a completion or feedback screen

The level itself does not decide what happens next.

This architecture ensures modularity, supports parallel development, and allows new levels to be integrated without modifying existing game flow logic.

Concept Art & Visual Design

This section looks at the concept art and character designs created during the early stages of the game. Due to the short timeline for development, we used AI-generated assets for the visuals. If the game were to be developed further beyond this course, we would work with an artist and replace these assets with original artwork.

Alvin (Early Concept):

This character model was inspired by the Na'vi people from the Avatar franchise.



Alvin's Ship:



These models are early concepts intended to represent what we plan to develop, and some aspects may change between now and the final product. The purpose of this section is to give the audience a general idea of what we envisioned at each stage of development.

Gameplay Mechanics Overview

Acid Rain
Vslvmvvv,vsv

Trash Pickup
Ncdvv

Desert Dig
Vewrvnvrkv

Space Junk
Yvbj

Penguin Slide
Venrvjrvn

User Stories

ID.	User Story
US-1	As a player, I want to log in to the game so that I can access my account and play.
US-2	As a player, I want to track my level progress on the level selection screen so that I know what I still need to complete.
US-3	As a new player, I want to be able to create an account so that I can play the game.
US-4	As a player, I want to be able to delete my account when I am tired of playing the game.
US-5	As a player, I want short cutscenes to be played so that I can feel fully immersed in the story.
US-6	As a player, I want to be able to select and start an unlocked level from the level selection screen.
US-7	As a player, I want to be able to collect stars within a level so that I can fill my collection and help save the Earth.
US-8	As a player, I want the number of stars I collect in each level to be saved so that I do not have to restart my progress each time I log in.
US-9	As a player, I want the progress I make in each level to contribute to the Earth's overall health total.
US-10	As a player, I want to receive feedback when I complete a level so that I understand my impact and what I accomplished.

Functional Requirements

ID.	Functional Requirement	US. ID
FR-1	The system will allow users to log in using a valid username and password.	US-1
FR-2	The system will display the player's level progress to make it easier for players to track their progress.	US-2
FR-3	The system will allow new users to create an account.	US-3
FR-4	The system will allow users to delete their account.	US-4
FR-5	The system will play short cutscenes at appropriate points in the game.	US-5
FR-6	The system will allow players to select and start an unlocked level from the level selection screen.	US-6
FR-7	The system will allow players to collect stars during gameplay within a level.	US-7
FR-8	The system will save the number of stars collected for each level.	US-8
FR-9	The system will update the Earth's health total based on player progress in completed levels.	US-9
FR-10	The system will display feedback to the player upon completion of a level.	US-10

Non Functional Requirements

ID.	NF	Description	FR. ID
NF-1	Security	The Login process required users to enter a valid username and password.	FR-1
NF-2	Usability	The system shall display the player's collected stars for each level on the level selection screen.	FR-2
NF-3	Reliability	The system prevents account creation if any of the required fields (email, username, password, or date of birth) are missing or invalid.	FR-3
NF-4	Security	Account deletion shall require password confirmation to prevent unauthorized access	FR-4
NF-5	Accessibility	Cutscenes shall include subtitles to ensure accessibility for players with hearing impairments.	FR-5

Level Design Breakdown

Acid Rain

River clearing

Desert Dig

Space Junk

Penguin Aid

Data Management & Persistence

Assumptions & Constraints

Conclusion